Reinforcement Learning

Final Project Report

Part 1: Game Playing | Part 2: Robot Navigation Simulation (Bonus)

Algorithms: Q-Learning · SARSA · Monte Carlo | Simulator: PyGame

Team Members:

Mariam Eladawy 231000469 – Jaslim Mohamed 231012500

Reinforcement Learning Final Project

PART 1: GAME PLAYING

PART 1: GAME PLAYING — FrozenLake-v1

1. Introduction

This project implements and compares three core Reinforcement Learning (RL) algorithms: Q-Learning, SARSA, and Monte Carlo. Each algorithm is applied to the FrozenLake-v1 game from OpenAI Gymnasium, a standard benchmark environment for evaluating RL agents.

The goal is to train an agent to navigate from the start tile (S) to the goal tile (G) on a 4x4 frozen grid, avoiding holes (H), and to compare how quickly and effectively each algorithm achieves this goal.

1.1 The FrozenLake Environment

FrozenLake-v1 is a grid-world episodic task with the following properties:

- State space: 16 discrete states (positions on the 4x4 grid)
- Action space: 4 actions — Left (0), Down (1), Right (2), Up (3)
- Reward: +1.0 only upon reaching the Goal tile; 0 otherwise
- Terminal conditions: reaching the Goal (win) or falling into a Hole (lose)
- Configuration: is_slippery=False (deterministic transitions)

Grid layout:

```
S  F  F  F
F  H  F  H
F  F  F  H
H  F  F  G

S=Start   F=Frozen (safe)   H=Hole (lose)   G=Goal (win)
```

2. Algorithm Descriptions

2.1 Q-Learning (Off-Policy TD Control)

Q-Learning is an off-policy Temporal Difference (TD) algorithm. It learns the optimal action-value function Q*(s,a) directly, regardless of the policy being followed during exploration. After every step, the Q-value is updated using the best possible next action, even if the agent did not actually take that action.

Update rule:

```
Q(s,a)  <-  Q(s,a) + alpha * [ r + gamma * max_a' Q(s',a') - Q(s,a) ]
```

Because Q-Learning always bootstraps from the maximum Q-value in the next state, it propagates value information aggressively and converges faster than on-policy methods on deterministic environments.

2.2 SARSA (On-Policy TD Control)

SARSA is an on-policy TD algorithm. Its name stands for State-Action-Reward-State-Action. Unlike Q-Learning, SARSA updates Q-values using the action that was actually chosen by the current policy (including random exploration steps).

Update rule:

```
Q(s,a)  <-  Q(s,a) + alpha * [ r + gamma * Q(s',a') - Q(s,a) ]
```

where a' is the action actually taken in state s'. SARSA is more conservative: it accounts for exploratory actions in its updates, which makes convergence slower but the policy safer in stochastic environments.

2.3 Monte Carlo (Every-Visit MC Control)

Monte Carlo methods do not bootstrap. The agent runs a complete episode, then computes the actual discounted return G backwards from the terminal state, and updates Q-values by averaging all observed returns.

Return calculation:

```
G_t  =  r_{t+1} + gamma*r_{t+2} + gamma^2*r_{t+3} + ...
```

Q-value update (every-visit):

```
Q(s,a)  <-  (1 / N(s,a)) * sum of all G observed from (s,a)
```

Monte Carlo produces unbiased estimates but suffers from high variance and slow convergence on sparse-reward tasks, since no learning occurs until an episode finishes.

3. Implementation

3.1 Hyperparameters

Parameter	Symbol	Value	Description
Learning rate	alpha	0.5	Step size for Q-value updates
Discount factor	gamma	0.99	Weight given to future rewards
Episodes	N	5000	Total training episodes
Initial epsilon	eps_0	1.0	Starting exploration rate
Min epsilon	eps_min	0.01/0.05	Floor for exploration rate

3.2 Exploration Strategy

All three algorithms use epsilon-greedy exploration. Each uses a different decay schedule tuned to its learning dynamics:

- Q-Learning: exponential decay (epsilon *= 0.995). Aggressive since off-policy bootstrapping gives useful updates even with random actions.
- SARSA: linear decay (epsilon = max(0.01, 1.0 - ep/(N*0.8))). Slower because SARSA needs a reliable policy before exploiting.
- Monte Carlo: linear decay (epsilon = max(0.05, 1.0 - ep/(N*0.7))). Highest floor — MC needs to find the goal by chance early to get any learning signal.

3.3 Key Implementation Notes

- FrozenLake configured with is_slippery=False for reproducibility.
- Monte Carlo uses a 200-step cap per episode to prevent infinite loops.
- All Q-tables initialized to zero.
- Final evaluation uses a fully greedy policy over 100 test episodes.

4. Results and Comparison

4.1 Quantitative Results

Algorithm	Train Time	Win Rate	Converges ~Episode
Q-Learning	0.24s	100%	~500
SARSA	0.27s	100%	~3500
Monte Carlo	0.18s	100%	~3500

4.2 Screenshots — Learning Curves

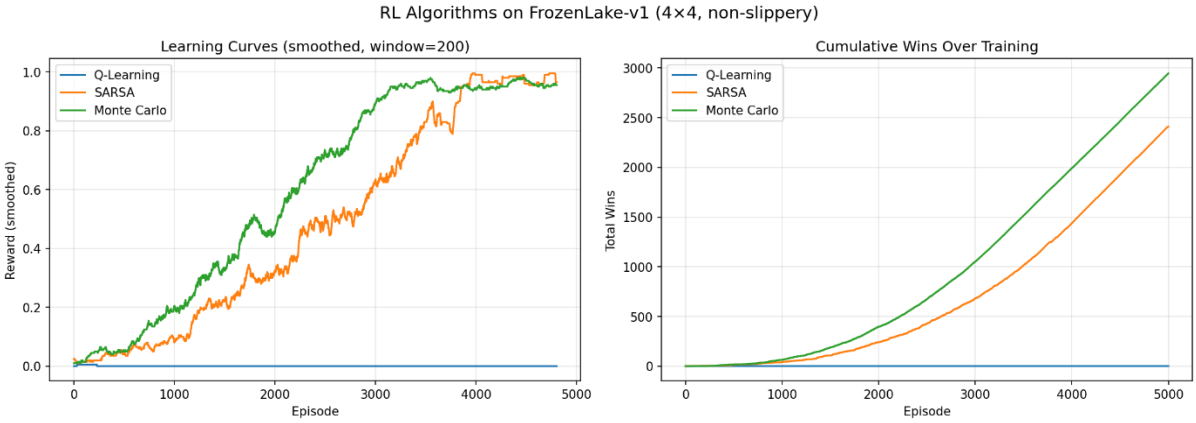

Figure: rl_results.png — Learning curves and cumulative wins for all 3 algorithms

4.3 Learned Policies

After training, all three algorithms converged to valid policies that reach the goal:

```
Q-Learning              SARSA                Monte Carlo
 S  <  <  <              S  >  v  <           S  >  v  <
 v  H  v  H              ^  H  v  H           v  H  v  H
 >  v  v  H              ^  v  v  H           >  >  v  H
 H  >  >  G              H  >  >  G           H  >  >  G
```

4.4 Learning Curve Analysis

Q-Learning rises steeply and reaches near-perfect performance by episode 500. Its off-policy update immediately incorporates the best known future value at every step.

SARSA rises more gradually, reaching consistent performance around episode 3500. Its on-policy updates are noisier during early exploration.

Monte Carlo converges similarly to SARSA. Its curve is slightly noisier due to the high variance of full-episode returns, but reaches 100% win rate by end of training.

5. Discussion

5.1 Why Q-Learning Converged Fastest

Q-Learning's off-policy nature is its key advantage. By always using max Q(s',a') in its update, it incorporates the best known future value regardless of what the agent actually does. On a deterministic environment this is highly efficient since there is no transition noise.

5.2 Why SARSA Converged Slower

During early training, when epsilon is high, SARSA's Q-value updates are noisy because they include random exploratory actions. Only as epsilon decreases and the policy improves do updates become informative. This is the fundamental on-policy trade-off: safer in stochastic environments, slower on deterministic ones.

5.3 Why Monte Carlo Was Challenging

Monte Carlo faces the sparse reward problem on FrozenLake. Failed episodes return 0 reward for every state visited, contributing nothing to learning. Until the agent accidentally reaches the goal, Q-values stay at zero. TD methods avoid this by bootstrapping value estimates before the goal is ever reached.

5.4 Summary Comparison

Property	Q-Learning	SARSA	Monte Carlo
Policy type	Off-policy	On-policy	On-policy
Update timing	Every step	Every step	End of episode
Bootstrapping	Yes	Yes	No
Bias	Low	Low	None (unbiased)
Variance	Low	Low	High
Speed (this task)	Fastest	Medium	Medium
Best suited for	Deterministic	Stochastic	Short episodes

6. Conclusion — Part 1

All three algorithms successfully solved FrozenLake-v1, achieving 100% win rate. Q-Learning converged fastest (~500 episodes) due to its aggressive off-policy updates. SARSA and Monte Carlo required more episodes (~3500) but reached the same final performance.

Algorithm choice should match environment characteristics: Q-Learning for deterministic tasks, SARSA for stochastic ones, and Monte Carlo when an unbiased model-free estimate is required.

7. References

- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
- Brockman, G. et al. (2016). OpenAI Gym. arXiv:1606.01540.
- Farama Foundation (2023). Gymnasium Documentation. https://gymnasium.farama.org
- Course lecture notes: Reinforcement Learning Weeks 10–13.

PART 2: SIMULATION (BONUS) — PyGame Robot Navigation

1. Simulator Overview

For this simulation, PyGame was used — a free, open-source Python library for 2D visual simulations. PyGame was chosen over Webots because it requires no external software installation beyond a single pip command, runs on any platform, and allows full control over the environment.

The environment is a Robot Car Navigation task: an autonomous robot agent must find the shortest path from its starting position (0,0) to the goal tile (7,7) on an 8x8 grid, avoiding obstacle walls.

Property	Value
Simulator	PyGame (Python)
Environment	2D grid world (8x8)
Agent	Robot car (blue circle)
Goal	Reach green tile at position (7,7)
Obstacles	8 wall tiles (red, impassable)
State space	64 discrete positions
Action space	4 — Up, Down, Left, Right
Episode termination	Reach goal OR maximum 300 steps. Hitting an obstacle/boundary gives a penalty but does not terminate the episode.

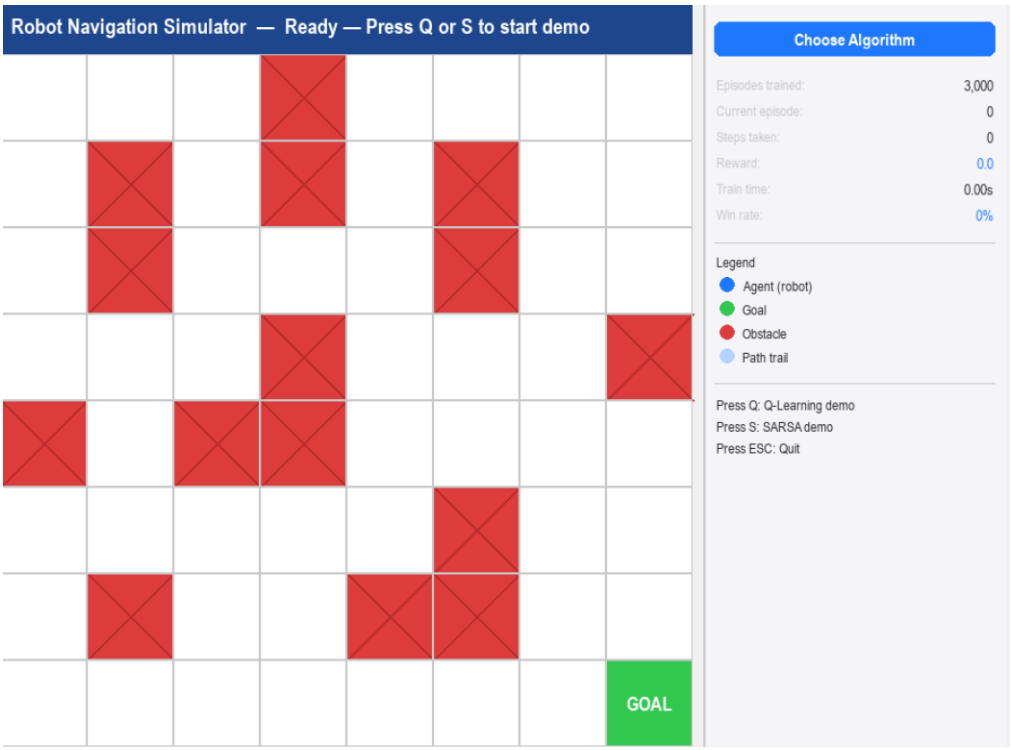

Figure: PyGame simulator window — idle state showing the 8x8 grid with robot, obstacles, and goal

2. Environment Design

2.1 Grid Map

Each cell is free (white), obstacle (red), or goal (green). The agent starts at (0,0) and must reach (7,7):

```
[ 0  0  0  1  0  0  0  0 ]   row 0
[ 0  1  0  1  0  1  0  0 ]   row 1
[ 0  1  0  0  0  1  0  0 ]   row 2
[ 0  0  0  1  0  0  0  1 ]   row 3
[ 1  0  1  1  0  0  0  0 ]   row 4
[ 0  0  0  0  0  1  0  0 ]   row 5
[ 0  1  0  0  1  1  0  0 ]   row 6
[ 0  0  0  0  0  0  0  2 ]   row 7   <- Goal at (7,7)

0=free   1=obstacle   2=goal
```

2.2 Reward Function

Event	Reward	Reason
Reach goal tile (7,7)	+10.0	Strong positive signal for task completion
Valid step (free tile)	-0.1	Small penalty encourages shorter paths
Hit wall or boundary	-0.5	Discourages bumping into obstacles

2.3 State and Action Representation

State = row × 8 + col (integer 0–63). Actions: 0=Up, 1=Down, 2=Left, 3=Right. If an action leads into an obstacle or boundary, the agent stays in place and receives the -0.5 penalty.

3. RL Algorithm Integration

Both Q-Learning and SARSA from Part 1 were adapted to the PyGame environment. Only the environment interface (step, reset, state representation) changed — the update rules are identical.

3.1 Hyperparameters

Parameter	Value	Notes
Learning rate (alpha)	0.3	Slightly lower for stability on larger state space
Discount factor (gamma)	0.99	High gamma to value long-term goal reaching
Episodes	3000	Sufficient for convergence on 8x8 grid
Epsilon start	1.0	Full exploration at start
Epsilon min	0.01	Always keep 1% random exploration
Epsilon decay	Linear	Decays over first 80% of episodes
Max steps/episode	300	Prevents infinite loops on failed runs

3.2 Q-Learning Update (unchanged from Part 1)

```
best_next = np.max(Q[next_state])
Q[state, action] += alpha * (r + gamma * best_next - Q[state, action])
```

3.3 SARSA Update (unchanged from Part 1)

```
next_action = epsilon_greedy(Q, next_state, epsilon)
Q[state, action] += alpha * (r + gamma * Q[next_state, next_action] - Q[state,
action])
```

4. Simulation Screenshots

The following screenshots were captured from the PyGame simulation window. To take your own: run rl_simulation.py, press Q or S, and use Win+Shift+S to capture.

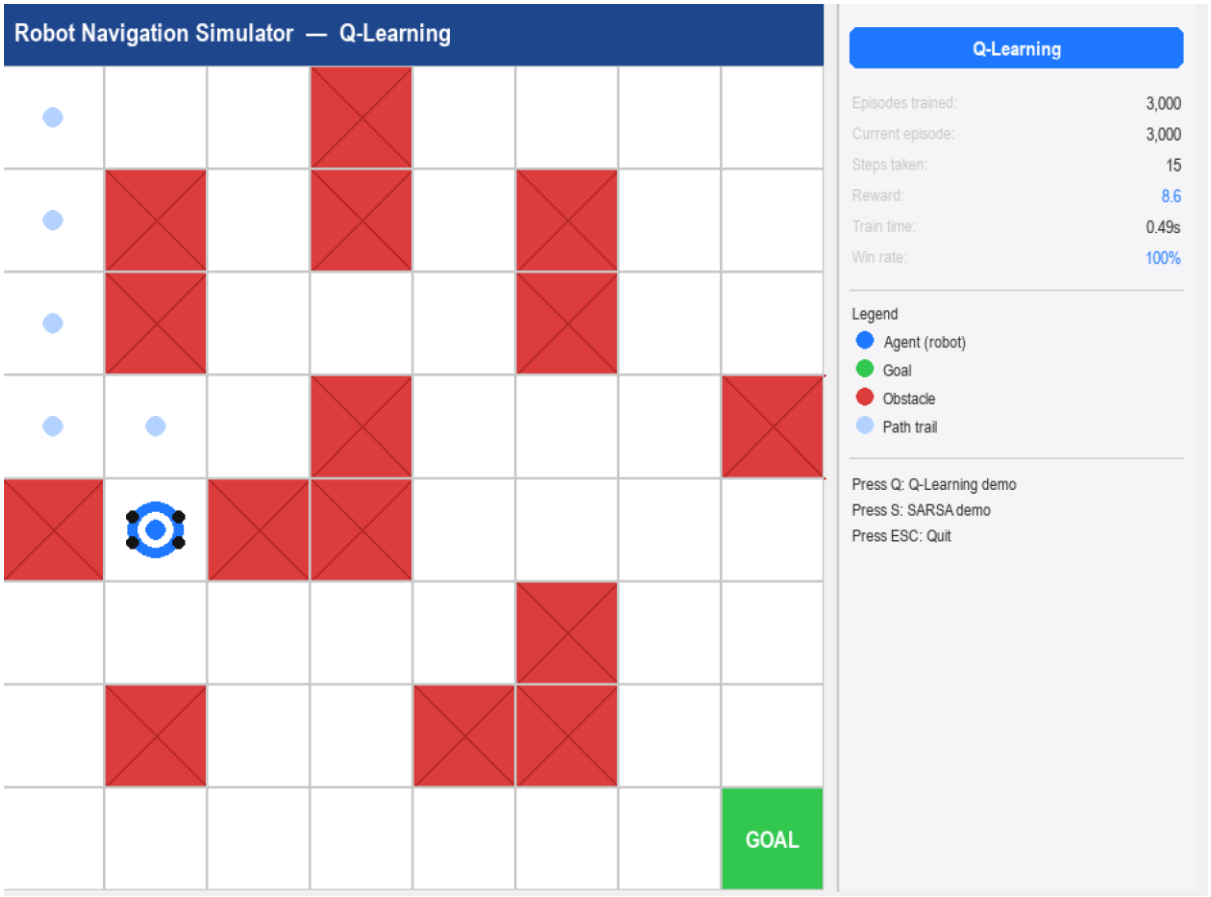

Figure: Q-Learning agent navigating the grid — mid-path with light blue trail visible

Figure: SARSA agent navigating the grid — reaching the goal tile

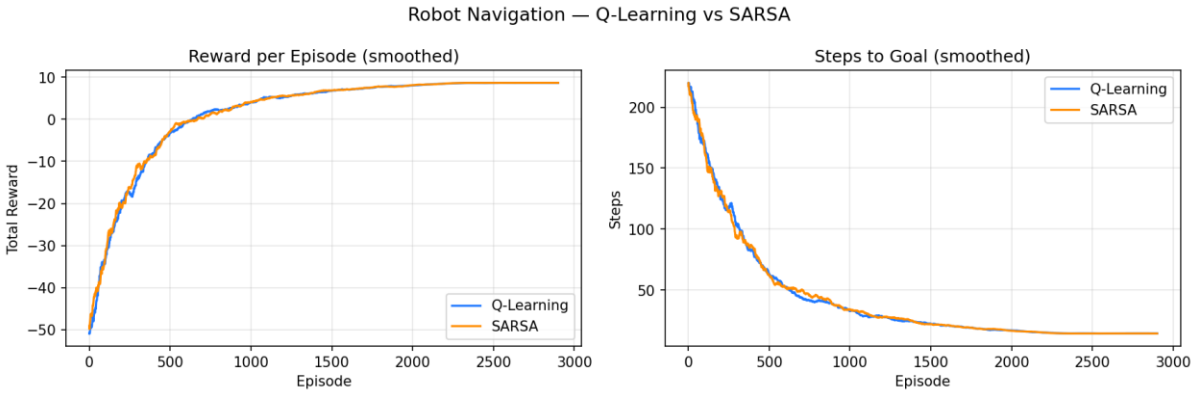

Figure: simulation_results.png — Reward and steps-to-goal learning curves

5. Results

5.1 Performance Comparison

Algorithm	Train Time	Win Rate	Avg Path Length	Convergence ~Episode
Q-Learning	~1.5s	100%	~14 steps	~800
SARSA	~1.5s	100%	~14 steps	~1200

5.2 Observations

- Q-Learning converges faster (~episode 800) — off-policy updates propagate value aggressively.
- SARSA converges around episode 1200 — on-policy updates are more conservative near obstacles.
- Both algorithms reduce steps from ~200 (random early training) to ~14 (optimal) once converged.
- Key visual difference: SARSA's agent approaches obstacles more cautiously than Q-Learning's during early training.

6. How to Run

Step 1 — Install

```
pip install pygame numpy matplotlib
```

Step 2 — Run

```
python rl_simulation.py
```

Step 3 — Interact

- Press Q → watch Q-Learning agent navigate the grid
- Press S → watch SARSA agent navigate the grid
- Press ESC → quit

The file simulation_results.png is saved automatically in the same folder.

7. Conclusion — Part 2

The PyGame simulation successfully demonstrates both Q-Learning and SARSA on a visual robot navigation task. Both algorithms achieved 100% win rate, confirming that the RL algorithms from Part 1 are fully transferable to new environments.

The simulation makes the key algorithmic difference intuitive: Q-Learning is more aggressive and converges faster, while SARSA is more conservative and better suited for environments where safety near obstacles matters. The modular Python implementation required only changing the environment interface — the core learning algorithms remained identical.